```
import sqlite3
import hashlib
import os
from datetime import date, datetime

DB = "gym_management.db"


# ---------- DATABASE CONNECTION ----------
def connect():
    con = sqlite3.connect(DB)
    con.row_factory = sqlite3.Row
    con.execute("PRAGMA foreign_keys = ON")
    return con


# ---------- PASSWORD SECURITY ----------
def make_hash(password, salt=None):
    salt = salt or os.urandom(16).hex()
    hashed = hashlib.pbkdf2_hmac(
        "sha256", password.encode(),
        bytes.fromhex(salt), 120000
    ).hex()
    return salt, hashed


def verify_password(password, salt, hashed):
    return make_hash(password, salt)[1] == hashed


# ---------- DATABASE SETUP ----------
def setup():
    with connect() as con:
        con.executescript("""
        CREATE TABLE IF NOT EXISTS users(
            user_id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            salt TEXT NOT NULL,
            password_hash TEXT NOT NULL,
            role TEXT NOT NULL CHECK(role IN ('Admin','Staff'))
        );

        CREATE TABLE IF NOT EXISTS members(
            member_id INTEGER PRIMARY KEY AUTOINCREMENT,
            full_name TEXT NOT NULL,
            phone TEXT UNIQUE NOT NULL,
            email TEXT UNIQUE,
            join_date TEXT NOT NULL,
            status TEXT NOT NULL DEFAULT 'Active',
            created_by INTEGER,
            FOREIGN KEY(created_by) REFERENCES users(user_id)
                ON DELETE SET NULL
        );

        CREATE TABLE IF NOT EXISTS plans(
            plan_id INTEGER PRIMARY KEY AUTOINCREMENT,
            plan_name TEXT UNIQUE NOT NULL,
            duration_days INTEGER NOT NULL CHECK(duration_days > 0),
            price REAL NOT NULL CHECK(price >= 0),
            description TEXT
        );

        CREATE TABLE IF NOT EXISTS memberships(
            membership_id INTEGER PRIMARY KEY AUTOINCREMENT,
            member_id INTEGER NOT NULL,
            plan_id INTEGER NOT NULL,
            start_date TEXT NOT NULL,
            end_date TEXT NOT NULL,
```

```
            payment_status TEXT NOT NULL DEFAULT 'Paid',
            FOREIGN KEY(member_id) REFERENCES members(member_id)
                ON DELETE CASCADE,
            FOREIGN KEY(plan_id) REFERENCES plans(plan_id)
                ON DELETE RESTRICT
        );
        """)

        if con.execute("SELECT COUNT(*) FROM users").fetchone()[0] == 0:
            for username, password, role in [
                ("admin", "Admin@123", "Admin"),
                ("staff", "Staff@123", "Staff")
            ]:
                salt, hashed = make_hash(password)
                con.execute("""
                    INSERT INTO users
                    (username, salt, password_hash, role)
                    VALUES (?, ?, ?, ?)
                """, (username, salt, hashed, role))

        if con.execute("SELECT COUNT(*) FROM plans").fetchone()[0] == 0:
            con.executemany("""
                INSERT INTO plans
                (plan_name, duration_days, price, description)
                VALUES (?, ?, ?, ?)
            """, [
                ("Monthly", 30, 1500, "30-day membership"),
                ("Quarterly", 90, 4000, "90-day membership"),
                ("Annual", 365, 12000, "365-day membership")
            ])

        if con.execute("SELECT COUNT(*) FROM members").fetchone()[0] == 0:
            admin_id = con.execute(
                "SELECT user_id FROM users WHERE username='admin'"
            ).fetchone()["user_id"]

            con.execute("""
                INSERT INTO members
                (full_name, phone, email, join_date, created_by)
                VALUES (?, ?, ?, ?, ?)
            """, (
                "Anu Thomas", "9000000001", "anu@example.com",
                date.today().isoformat(), admin_id
            ))

            member_id = con.execute(
                "SELECT member_id FROM members WHERE phone=?",
                ("9000000001",)
            ).fetchone()["member_id"]

            plan_id = con.execute(
                "SELECT plan_id FROM plans WHERE plan_name='Monthly'"
            ).fetchone()["plan_id"]

            start = date.today()
            end = date.fromordinal(start.toordinal() + 29)

            con.execute("""
                INSERT INTO memberships
                (member_id, plan_id, start_date, end_date, payment_status)
                VALUES (?, ?, ?, ?, ?)
            """, (
                member_id, plan_id, start.isoformat(),
                end.isoformat(), "Paid"
            ))


    # ---------- INPUT VALIDATION ----------
    def required(prompt):
```

```
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print("This field cannot be empty.")


def get_integer(prompt, minimum=1):
    while True:
        try:
            value = int(input(prompt).strip())
            if value >= minimum:
                return value
        except ValueError:
            pass
        print("Enter a valid whole number.")


def get_date(prompt):
    while True:
        try:
            return datetime.strptime(
                input(prompt).strip(), "%Y-%m-%d"
            ).date()
        except ValueError:
            print("Enter date as YYYY-MM-DD.")


# ---------- REGISTRATION AND LOGIN ----------
def register():
    print("\n--- Staff Registration ---")
    username = required("Username: ")
    password = required("Password (minimum 8 characters): ")

    if len(password) < 8:
        print("Password must contain at least 8 characters.")
        return

    salt, hashed = make_hash(password)

    try:
        with connect() as con:
            con.execute("""
                INSERT INTO users
                (username, salt, password_hash, role)
                VALUES (?, ?, ?, 'Staff')
            """, (username, salt, hashed))
        print("Staff registration successful.")

    except sqlite3.IntegrityError:
        print("Username already exists.")
    except sqlite3.Error as error:
        print("Database error:", error)


def login():
    username = required("Username: ")
    password = required("Password: ")

    try:
        with connect() as con:
            user = con.execute(
                "SELECT * FROM users WHERE username=?",
                (username,)
            ).fetchone()

        if user and verify_password(
            password, user["salt"], user["password_hash"]
        ):
```

```
            print("Login successful. Role:", user["role"])
            return dict(user)

        print("Incorrect username or password.")

    except sqlite3.Error as error:
        print("Database error:", error)

    return None


# ---------- MEMBER CRUD ----------
def view_members():
    with connect() as con:
        rows = con.execute(
            "SELECT * FROM members ORDER BY member_id"
        ).fetchall()

    print("\n--- GYM MEMBERS ---")
    if not rows:
        print("No members found.")
        return

    for m in rows:
        print(
            f"ID: {m['member_id']} | Name: {m['full_name']} | "
            f"Phone: {m['phone']} | Email: {m['email']} | "
            f"Joined: {m['join_date']} | Status: {m['status']}"
        )


def add_member(user):
    name = required("Full name: ")
    phone = required("Phone number: ")
    email = input("Email (optional): ").strip() or None
    join_date = get_date("Join date (YYYY-MM-DD): ").isoformat()

    try:
        with connect() as con:
            con.execute("""
                INSERT INTO members
                (full_name, phone, email, join_date, created_by)
                VALUES (?, ?, ?, ?, ?)
            """, (name, phone, email, join_date, user["user_id"]))

        print("Member added successfully.")

    except sqlite3.IntegrityError:
        print("Phone number or email already exists.")
    except sqlite3.Error as error:
        print("Database error:", error)


def update_member():
    view_members()
    member_id = get_integer("Member ID to update: ")

    try:
        with connect() as con:
            member = con.execute(
                "SELECT * FROM members WHERE member_id=?",
                (member_id,)
            ).fetchone()

            if not member:
                print("Member not found.")
                return

            name = input(
```

```
                f"Name [{member['full_name']}]: "
            ).strip() or member["full_name"]

            phone = input(
                f"Phone [{member['phone']}]: "
            ).strip() or member["phone"]

            email = input(
                f"Email [{member['email'] or ''}]: "
            ).strip() or member["email"]

            status = input(
                f"Status Active/Inactive [{member['status']}]: "
            ).strip().title() or member["status"]

            if status not in ("Active", "Inactive"):
                print("Invalid status.")
                return

            con.execute("""
                UPDATE members
                SET full_name=?, phone=?, email=?, status=?
                WHERE member_id=?
            """, (name, phone, email, status, member_id))

        print("Member updated successfully.")

    except sqlite3.IntegrityError:
        print("Phone or email already belongs to another member.")
    except sqlite3.Error as error:
        print("Database error:", error)


def delete_member():
    view_members()
    member_id = get_integer("Member ID to delete: ")

    confirm = input("Confirm deletion? (yes/no): ").strip().lower()
    if confirm not in ("yes", "y"):
        print("Deletion cancelled.")
        return

    try:
        with connect() as con:
            cursor = con.execute(
                "DELETE FROM members WHERE member_id=?",
                (member_id,)
            )

        if cursor.rowcount:
            print("Member deleted.")
        else:
            print("Member not found.")

    except sqlite3.Error as error:
        print("Database error:", error)


# ---------- MEMBERSHIP PLANS ----------
def view_plans():
    with connect() as con:
        rows = con.execute(
            "SELECT * FROM plans ORDER BY plan_id"
        ).fetchall()

    print("\n--- MEMBERSHIP PLANS ---")
    for p in rows:
        print(
            f"ID: {p['plan_id']} | {p['plan_name']} | "
```

```
            f"{p['duration_days']} days | Rs. {p['price']:.2f}"
        )


def add_plan():
    name = required("Plan name: ")
    days = get_integer("Duration in days: ")

    while True:
        try:
            price = float(input("Price in INR: "))
            if price >= 0:
                break
        except ValueError:
            pass
        print("Enter a valid non-negative price.")

    description = input("Description: ").strip()

    try:
        with connect() as con:
            con.execute("""
                INSERT INTO plans
                (plan_name, duration_days, price, description)
                VALUES (?, ?, ?, ?)
            """, (name, days, price, description))

        print("Plan added successfully.")

    except sqlite3.IntegrityError:
        print("Plan name already exists.")
    except sqlite3.Error as error:
        print("Database error:", error)


def update_plan():
    view_plans()
    plan_id = get_integer("Plan ID to update: ")

    try:
        with connect() as con:
            plan = con.execute(
                "SELECT * FROM plans WHERE plan_id=?",
                (plan_id,)
            ).fetchone()

            if not plan:
                print("Plan not found.")
                return

            name = input(
                f"Name [{plan['plan_name']}]: "
            ).strip() or plan["plan_name"]

            days_input = input(
                f"Duration [{plan['duration_days']}]: "
            ).strip()

            price_input = input(
                f"Price [{plan['price']}]: "
            ).strip()

            days = int(days_input) if days_input else plan["duration_days"]
            price = float(price_input) if price_input else plan["price"]

            if days <= 0 or price < 0:
                print("Invalid duration or price.")
                return
```

```
            con.execute("""
                UPDATE plans
                SET plan_name=?, duration_days=?, price=?
                WHERE plan_id=?
            """, (name, days, price, plan_id))

        print("Plan updated successfully.")

    except ValueError:
        print("Invalid numeric value.")
    except sqlite3.IntegrityError:
        print("Plan name already exists.")
    except sqlite3.Error as error:
        print("Database error:", error)


def delete_plan():
    view_plans()
    plan_id = get_integer("Plan ID to delete: ")

    if input("Confirm deletion? (yes/no): ").strip().lower() not in ("yes", "y"):
        print("Deletion cancelled.")
        return

    try:
        with connect() as con:
            cursor = con.execute(
                "DELETE FROM plans WHERE plan_id=?",
                (plan_id,)
            )

        print("Plan deleted." if cursor.rowcount else "Plan not found.")

    except sqlite3.IntegrityError:
        print("Cannot delete a plan currently assigned to a member.")
    except sqlite3.Error as error:
        print("Database error:", error)


# ---------- MEMBERSHIP MANAGEMENT ----------
def assign_membership():
    view_members()
    member_id = get_integer("Member ID: ")

    view_plans()
    plan_id = get_integer("Plan ID: ")

    start = get_date("Start date (YYYY-MM-DD): ")
    payment = input("Payment status Paid/Pending [Paid]: ").strip().title() or "Paid"

    if payment not in ("Paid", "Pending"):
        print("Invalid payment status.")
        return

    try:
        with connect() as con:
            member = con.execute(
                "SELECT * FROM members WHERE member_id=?",
                (member_id,)
            ).fetchone()

            plan = con.execute(
                "SELECT * FROM plans WHERE plan_id=?",
                (plan_id,)
            ).fetchone()

            if not member:
                print("Member ID not found.")
                return
```

```
            if not plan:
                print("Plan ID not found.")
                return

            end = date.fromordinal(
                start.toordinal() + plan["duration_days"] - 1
            )

            con.execute("""
                INSERT INTO memberships
                (member_id, plan_id, start_date, end_date, payment_status)
                VALUES (?, ?, ?, ?, ?)
            """, (
                member_id, plan_id, start.isoformat(),
                end.isoformat(), payment
            ))

        print("Membership assigned. End date:", end.isoformat())

    except sqlite3.Error as error:
        print("Database error:", error)


def view_memberships():
    with connect() as con:
        rows = con.execute("""
            SELECT ms.membership_id, m.full_name, p.plan_name,
                   ms.start_date, ms.end_date, ms.payment_status
            FROM memberships ms
            JOIN members m ON ms.member_id=m.member_id
            JOIN plans p ON ms.plan_id=p.plan_id
            ORDER BY ms.membership_id
        """).fetchall()

    print("\n--- MEMBERSHIP RECORDS ---")
    if not rows:
        print("No membership records found.")
        return

    for r in rows:
        print(
            f"ID: {r['membership_id']} | {r['full_name']} | "
            f"{r['plan_name']} | {r['start_date']} to {r['end_date']} | "
            f"Payment: {r['payment_status']}"
        )


def update_payment():
    view_memberships()
    membership_id = get_integer("Membership record ID: ")
    status = required("New payment status (Paid/Pending): ").title()

    if status not in ("Paid", "Pending"):
        print("Status must be Paid or Pending.")
        return

    try:
        with connect() as con:
            cursor = con.execute("""
                UPDATE memberships SET payment_status=?
                WHERE membership_id=?
            """, (status, membership_id))

        print("Payment updated." if cursor.rowcount else "Record not found.")

    except sqlite3.Error as error:
        print("Database error:", error)
```

```
# ---------- ADMIN MENU ----------
def admin_menu(user):
    while True:
        print("""
===== ADMIN MENU =====
1. View members
2. Add member
3. Update member
4. Delete member
5. View plans
6. Add plan
7. Update plan
8. Delete plan
9. Assign membership
10. View memberships
11. Update payment status
12. Logout
""")
        choice = input("Choose an option: ").strip()

        actions = {
            "1": view_members,
            "2": lambda: add_member(user),
            "3": update_member,
            "4": delete_member,
            "5": view_plans,
            "6": add_plan,
            "7": update_plan,
            "8": delete_plan,
            "9": assign_membership,
            "10": view_memberships,
            "11": update_payment
        }

        if choice in actions:
            try:
                actions[choice]()
            except (ValueError, sqlite3.Error) as error:
                print("Could not complete action:", error)
        elif choice == "12":
            print("Logged out.")
            return
        else:
            print("Invalid menu selection.")


# ---------- STAFF MENU ----------
def staff_menu(user):
    while True:
        print("""
===== STAFF MENU =====
1. View members
2. Add member
3. View plans
4. Assign membership
5. View memberships
6. Update payment status
7. Logout
""")
        choice = input("Choose an option: ").strip()

        actions = {
            "1": view_members,
            "2": lambda: add_member(user),
            "3": view_plans,
            "4": assign_membership,
            "5": view_memberships,
            "6": update_payment
```

```
        }

        if choice in actions:
            try:
                actions[choice]()
            except (ValueError, sqlite3.Error) as error:
                print("Could not complete action:", error)
        elif choice == "7":
            print("Logged out.")
            return
        else:
            print("Invalid menu selection.")


# ---------- MAIN MENU ----------
def main():
    setup()

    while True:
        print("""
===== GYM MEMBERSHIP MANAGEMENT SYSTEM =====
1. Register staff
2. Login
3. Exit
""")
        choice = input("Choose an option: ").strip()

        if choice == "1":
            register()
        elif choice == "2":
            user = login()
            if user:
                if user["role"] == "Admin":
                    admin_menu(user)
                else:
                    staff_menu(user)
        elif choice == "3":
            print("Thank you for using the system.")
            break
        else:
            print("Invalid selection. Choose 1, 2, or 3.")


if __name__ == "__main__":
    try:
        main()
    except (KeyboardInterrupt, EOFError):
        print("\nApplication closed safely.")
```